

Е2Е-тесты для Django-application-e2e

Тест-кейсы на русском языке, разбор каждого автотеста и как их запускать

Проект: github.com/Senpai520120/e2e_tests

Тестируемое приложение: github.com/Senpai520120/Django-application-e2e

Стек: Python 3.11+, pytest, Playwright (pytest-playwright)

Английская версия тест-кейсов — [docs/test-cases.md](#) и [issue](#) с меткой `test-case` на GitHub. Этот документ — перевод и учебные пояснения к ним.

1. Как запустить тесты

Шаг 1. Запустить приложение

В папке репозитория **приложения** (Django-application-e2e):

```
copy .env.example .env # и вписать в SECRET_KEY любую длинную строку
docker compose up --build
```

Приложение откроется на `http://127.0.0.1:8000`. Демо-аккаунты создаются сами: `demo_admin`, `demo_user`, `demo_inactive`, пароль у всех `demo-password-123`. После `git pull` в приложении всегда запускайте с `--build`, иначе Docker поднимет старую версию кода.

Шаг 2. Установить тесты (один раз)

В папке репозитория **тестов** (e2e_tests):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
playwright install chromium
```

Признак того, что окружение активно: в начале строки терминала написано (`.venv`).

Шаг 3. Запуск

Команда	Что делает
<code>pytest</code>	все 34 теста
<code>pytest -v</code>	то же, но с именем каждого теста и результатом
<code>pytest tests/test_login.py</code>	только один файл
<code>pytest -k test_05</code>	один тест по номеру (ищет подстроку в имени)
<code>pytest -k "test_01 or test_02"</code>	несколько тестов
<code>pytest --headed --slowmo 500</code>	показать браузер и замедлить каждое действие на 0,5 с — удобно, чтобы посмотреть глазами
<code>pytest -x</code>	остановиться на первом упавшем тесте
<code>pytest --base-url http://127.0.0.1:8001</code>	запустить против другой копии приложения

Если тест упал

Для упавшего теста Playwright сохраняет трейс, скриншот и видео в папку `test-results/`. Трейс — это запись каждого шага с «фотографией» страницы, его открывают так:

```
playwright show-trace test-results\<папка теста>\trace.zip
```

В VS Code тесты можно запускать и из панели **Testing** (иконка колбы слева): там же видно, какие прошли, а какие нет.

2. Как устроен проект

Файл / папка	Что внутри
tests/	сами тесты: ЧТО проверяем. Один тест = один тест-кейс, номер теста = номер кейса (test_05_... ↔ TC-05)
pages/	Page Object: КАК работать со страницей — где поля, кнопки и какие действия на ней можно сделать. Одна страница = один файл
conftest.py	фикстуры: подготовка перед тестом и уборка после (вход в систему, новый пользователь, пустая папка)
config.py	настройки и демо-аккаунты, всё можно поменять переменными окружения
helpers.py	unique_name() — уникальные имена, sign_in() — вход
pytest.ini	настройки запуска: адрес приложения, браузер, трейсы
tools/	test_cases.json (тест-кейсы) и скрипт, который делает из них issue на GitHub
docs/	test-cases.md (кейсы на английском) и этот PDF

Зачем делить на tests и pages

Если в приложении переименуют кнопку «Sign in», вы меняете одну строку в pages/login_page.py, а не десять тестов. Поэтому в тестах нет локаторов, а в Page Object нет проверок (expect): тест решает, что проверить, страница знает, как до этого добраться.

Как читать тест

Каждый тест построен по схеме «подготовка → действие → проверка» (Arrange → Act → Assert), блоки отделены пустой строкой:

```
def test_03_wrong_password_is_rejected(page: Page):
    """TC-03: sign-in with a wrong password shows an error."""
    login_page = LoginPage(page).open()  # подготовка

    login_page.sign_in("demo_user", "wrong-password")  # действие

    expect(login_page.error_message).to_be_visible()  # проверка
    expect(page).to_have_url(re.compile(r"/login/$"))
```

- Первая строка в кавычках (docstring) — номер и смысл кейса.
- expect(...) — проверка элемента страницы. Она **сама ждёт** до 10 секунд, пока условие выполнится, поэтому в тестах нет и не должно быть time.sleep().
- assert — обычная проверка Python, для значений, которые мы уже достали со страницы (строки, числа, содержимое файла).

Локаторы

Локатор — это «адрес» элемента на странице. Используем локаторы по роли и подписи, как их видит пользователь, а не CSS-классы:

- get_by_role("button", name="Sign in") — кнопка с текстом «Sign in»;
- get_by_label("Username") — поле, подписанное «Username»;
- get_by_text("Folder is empty.") — элемент с этим текстом;
- exact=True — совпадение целиком, а не по части текста.

3. Фикстуры

Фикстура — функция, которая готовит что-то для теста. Тест просит фикстуру, просто указав её имя в аргументах: `def test_06_user_signs_out(user_page):`. `pytest` видит имя `user_page`, находит фикстуру в `conftest.py`, выполняет её и передаёт результат в тест.

Если в фикстуре есть `yield`, она работает в две половины: код до `yield` готовит данные, потом идёт тест, потом выполняется код после `yield` — уборка. Уборка выполнится, даже если тест упал.

Фикстура	Откуда	Что даёт тесту
<code>page</code>	<code>pytest-playwright</code>	чистая вкладка браузера, никто не вошёл
<code>new_context</code>	<code>pytest-playwright</code>	функция: открыть ещё один отдельный браузер (свои cookie, своя сессия)
<code>tmp_path</code>	<code>pytest</code>	временная папка, удаляется сама
<code>admin_page</code>	<code>conftest.py</code>	вкладка, вошедшая как <code>demo_admin</code>
<code>user_page</code>	<code>conftest.py</code>	вкладка, вошедшая как <code>demo_user</code>
<code>new_user</code>	<code>conftest.py</code>	новый пользователь с уникальным именем (создан через админку). После теста — отключается
<code>files_page</code>	<code>conftest.py</code>	файловый менеджер под админом, открыт в корне
<code>workspace</code>	<code>conftest.py</code>	пустая папка теста, уже открыта; имя папки — в переменной. После теста — удаляется
<code>make_file</code>	<code>conftest.py</code>	функция: создать текстовый файл и получить путь к нему

Фикстуры могут зависеть друг от друга. Например, тест просит `workspace` → ей нужен `files_page` → ему нужен `admin_page` → ему нужен `page`. `pytest` сам выстроит эту цепочку.

Почему `workspace` подчеркнут серым

В тестах файлов `workspace` стоит в аргументах, но в коде теста может не использоваться. VS Code подсвечивает это как «не используется». Это нормально: тесту нужна не переменная, а то, что фикстура сделала — создала и открыла пустую папку, а после теста её удалит.

Почему тесты не мешают друг другу

- Каждый пользователь, папка и файл получает уникальное имя (`unique_name`).
- Тесты, которые меняют роли или отключают пользователя, работают с новым пользователем из `new_user`, а не с общим `demo_user`.
- Файловые тесты работают каждый в своей папке `workspace`.
- Поэтому тесты можно запускать в любом порядке и по одному.

4. Тест-кейсы и автотесты

Для каждого кейса: перевод на русский, как работает автотест, код автотеста из репозитория и команда, которой его запустить отдельно.

ТС-01. Аноним попадает на страницу входа

English: Anonymous user is sent to the sign-in page

Предусловия	Пользователь не вошёл в систему
-------------	---------------------------------

Шаги

1. Открыть главную страницу /

Ожидаемый результат

Адрес становится /login/?next=/, виден заголовок «Sign in».

Как работает автотест

- Фикстура `page` даёт чистую вкладку браузера: никто не вошёл.
- `page.goto("/")` открывает главную. Адрес сервера подставляется из `--base-url` в `pytest.ini`, поэтому в тесте пишем только путь.
- `expect(page).to_have_url(re.compile(...))` проверяет адрес регулярным выражением: `\?` значит буквальное значение, `$` — конец строки.
- Вторая проверка: на странице виден заголовок «Sign in» из `LoginPage`.

Код: test_login.py

```
def test_01_anonymous_user_is_sent_to_sign_in(page: Page):
    """TC-01: an anonymous user who opens the home page is sent to sign in."""
    page.goto("/")

    expect(page).to_have_url(re.compile(r"/login/\?next=/$"))
    expect(LoginPage(page).heading).to_be_visible()
```

Запуск: `pytest -k test_01 -v`

ТС-02. Вход с верными логином и паролем

English: User signs in with valid credentials

Предусловия	Пользователь не вошёл. Есть аккаунт <code>demo_user / demo-password-123</code>
-------------	--

Шаги

1. Открыть `/login/`
2. Ввести логин `demo_user` и пароль `demo-password-123`
3. Нажать «Sign in»

Ожидаемый результат

Открывается главная с заголовком «You are signed in as demo_user», в списке ролей есть «user».

Как работает автотест

- `LoginPage(page).open()` создаёт Page Object страницы входа и сразу открывает `/login/`. Метод `open()` возвращает сам объект, поэтому его можно сохранить в переменную `login_page`.
- `login_page.sign_in(...)` заполняет два поля и жмёт кнопку. Логин и пароль берутся из `config.REGULAR_USER`, а не пишутся в тесте.
- `to_have_text` сравнивает заголовок целиком, а `role_badge("user")` ищет название роли внутри основной части страницы.

Код: test_login.py

```
def test_02_user_signs_in_with_valid_credentials(page: Page):
```

```

"""TC-02: a user signs in and sees their name and role."""
login_page = LoginPage(page).open()

login_page.sign_in(config.REGULAR_USER.username, config.REGULAR_USER.password)

home_page = HomePage(page)
expect(home_page.heading).to_have_text("You are signed in as demo_user")
expect(home_page.role_badge("user")).to_be_visible()
Запуск: pytest -k test_02 -v

```

ТС-03. Вход с неверным паролем отклоняется

English: Sign-in with a wrong password is rejected

Предусловия	Пользователь не вошёл
-------------	-----------------------

Шаги

1. Открыть /login/
2. Ввести логин demo_user и пароль wrong-password
3. Нажать «Sign in»

Ожидаемый результат

Пользователь остаётся на /login/, видна ошибка «Please enter a correct username and password».

Как работает автотест

- Негативный тест: вводим заведомо неверный пароль.
- login_page.error_message — локатор пункта списка ошибок с нужным текстом, описан в pages/login_page.py.
- Проверяем две вещи: ошибка видна и адрес остался /login/ (вход не случился).

Код: test_login.py

```

def test_03_wrong_password_is_rejected(page: Page):
    """TC-03: sign-in with a wrong password shows an error."""
    login_page = LoginPage(page).open()

    login_page.sign_in(config.REGULAR_USER.username, "wrong-password")

    expect(login_page.error_message).to_be_visible()
    expect(page).to_have_url(re.compile(r"/login/$"))
Запуск: pytest -k test_03 -v

```

ТС-04. Несуществующий логин даёт ту же ошибку, что и неверный пароль

English: Unknown username gets the same error as a wrong password

Предусловия	Пользователь не вошёл
-------------	-----------------------

Шаги

1. Войти как demo_user с неверным паролем и запомнить текст ошибки
2. Войти как no_such_user_here с любым паролем

Ожидаемый результат

Тексты ошибок полностью совпадают: по странице нельзя понять, какие логины существуют.

Как работает автотест

- Проверка безопасности: если бы тексты отличались, злоумышленник мог бы перебором узнать существующие логины.
- inner_text() достаёт текст элемента в обычную строку Python.

- Сравниваем строки обычным assert: здесь мы сравниваем два значения между собой, а не элемент страницы с ожиданием, поэтому expect не нужен.

Код: test_login.py

```
def test_04_unknown_username_gets_the_same_error(page: Page):
    """TC-04: an unknown username gets the same error as a wrong password.

    If the texts were different, an attacker could find out which usernames
    exist in the system.
    """
    login_page = LoginPage(page).open()

    login_page.sign_in(config.REGULAR_USER.username, "wrong-password")
    error_for_wrong_password = login_page.error_message.inner_text()

    login_page.sign_in("no_such_user_here", "wrong-password")
    error_for_unknown_user = login_page.error_message.inner_text()

    assert error_for_unknown_user == error_for_wrong_password
Запуск: pytest -k test_04 -v
```

ТС-05. Отключённый пользователь не может войти

English: Deactivated user cannot sign in

Предусловия	Пользователь не вошёл. Аккаунт demo_inactive отключён
-------------	---

Шаги

1. Открыть /login/
2. Ввести логин demo_inactive и пароль demo-password-123
3. Нажать «Sign in»

Ожидаемый результат

Пользователь остаётся на /login/, видна ошибка входа.

Как работает автотест

- Пароль верный, но аккаунт отключён, поэтому Django показывает ту же общую ошибку входа.
- Структура такая же, как в ТС-03, меняется только пользователь config.INACTIVE_USER.

Код: test_login.py

```
def test_05_deactivated_user_cannot_sign_in(page: Page):
    """TC-05: a switched-off user cannot sign in."""
    login_page = LoginPage(page).open()

    login_page.sign_in(config.INACTIVE_USER.username, config.INACTIVE_USER.password)

    expect(login_page.error_message).to_be_visible()
    expect(page).to_have_url(re.compile(r"/login/$"))
Запуск: pytest -k test_05 -v
```

ТС-06. Выход из системы

English: User signs out

Предусловия	Выполнен вход как demo_user
-------------	-----------------------------

Шаги

1. Нажать «Sign out» в шапке
2. Открыть главную /

Ожидаемый результат

Главная не открывается: адрес становится /login/?next=/.

Как работает автотест

- Фикстура `user_page` уже вошла как `demo_user` (см. `conftest.py`).
- `home_page.sign_out()` нажимает кнопку «Sign out» — она описана один раз в `BasePage`, потому что есть на каждой странице.
- После выхода снова открываем / и ждём перенаправления на вход.

Код: test_login.py

```
def test_06_user_signs_out(user_page: Page):
    """TC-06: after signing out the home page asks to sign in again."""
    home_page = HomePage(user_page).open()

    home_page.sign_out()
    user_page.goto("/")
```

```
expect(user_page).to_have_url(re.compile(r"/login/\?next=/$"))
```

Запуск: `pytest -k test_06 -v`

ТС-07. Ссылка «Files» в шапке открывает файловый менеджер

English: Header link "Files" opens the file manager

Предусловия	Выполнен вход как demo_user
-------------	-----------------------------

Шаги

1. Открыть главную /
2. Нажать «Files» в шапке

Ожидаемый результат

Адрес становится /files/, виден заголовок «Files».

Как работает автотест

- `header_link("Files")` ищет ссылку только внутри шапки (роль `banner`), чтобы не спутать её с такой же ссылкой где-то ещё на странице.
- Проверяем адрес и заголовок страницы файлов.

Код: test_login.py

```
def test_07_header_link_opens_file_manager(user_page: Page):
    """TC-07: the "Files" link in the header opens the file manager."""
    home_page = HomePage(user_page).open()

    home_page.header_link("Files").click()

    expect(user_page).to_have_url(re.compile(r"/files/$"))
    expect(FilesPage(user_page).heading).to_be_visible()
```

Запуск: `pytest -k test_07 -v`

ТС-08. Обычный пользователь получает «доступ запрещён» в админке

English: Regular user gets "access denied" on the admin panel

Предусловия	Выполнен вход как demo_user (без роли admin)
-------------	--

Шаги

1. Открыть /manage/

Ожидаемый результат

Вместо списка пользователей показана страница «403 — access denied».

Как работает автотест

- Сразу открываем адрес админки под обычным пользователем.
- Ищем заголовок «403 — access denied» прямо через `get_by_role`: эта страница встречается в одном тесте, отдельный Page Object для неё не нужен.

Код: test_access.py

```
def test_08_regular_user_gets_403_on_admin_panel(user_page: Page):
    """TC-08: a user without the admin role gets "access denied" on /manage/."""
    user_page.goto("/manage/")

    expect(user_page.get_by_role("heading", name="403 - access denied")).to_be_visible()
Запуск: pytest -k test_08 -v
```

ТС-09. Админ открывает админ-панель

English: Admin opens the admin panel

Предусловия	Выполнен вход как demo_admin
-------------	------------------------------

Шаги

1. Открыть /manage/

Ожидаемый результат

Открывается страница «Users» со списком, в нём есть demo_admin.

Как работает автотест

- Фикстура `admin_page` входит как `demo_admin`.
- `UsersPage(...).open()` открывает /manage/. Проверяем заголовок и строку таблицы с `demo_admin`.
- `row(username)` ищет строку, в которой есть ячейка ровно с этим именем (`exact=True`), чтобы `demo_user` не совпал с `demo_user01`.

Код: test_access.py

```
def test_09_admin_opens_admin_panel(admin_page: Page):
    """TC-09: an admin opens the user list at /manage/."""
    users_page = UsersPage(admin_page).open()

    expect(users_page.heading).to_be_visible()
    expect(users_page.row("demo_admin")).to_be_visible()
Запуск: pytest -k test_09 -v
```

ТС-10. Обычный пользователь не видит ссылок админки в шапке

English: Regular user does not see admin links in the header

Предусловия	Выполнен вход как demo_user
-------------	-----------------------------

Шаги

1. Открыть главную /
2. Посмотреть на ссылки в шапке

Ожидаемый результат

«Files» есть. «Users», «Roles» и «Audit» нет.

Как работает автотест

- `to_have_count(0)` — правильный способ проверить, что элемента нет. Playwright немного подождёт и убедится, что он так и не появился.
- Сначала проверяем, что «Files» есть: так мы убеждаемся, что шапка вообще загрузилась, и проверка «нет» не пройдёт случайно на пустой странице.

Код: test_access.py

```
def test_10_regular_user_does_not_see_admin_links(user_page: Page):
    """TC-10: a regular user has no "Users", "Roles" and "Audit" links in the header."""
    home_page = HomePage(user_page).open()

    expect(home_page.header_link("Files")).to_be_visible()
    expect(home_page.header_link("Users")).to_have_count(0)
    expect(home_page.header_link("Roles")).to_have_count(0)
    expect(home_page.header_link("Audit")).to_have_count(0)
Запуск: pytest -k test_10 -v
```

ТС-11. Аноним не может открыть файловый менеджер

English: Anonymous user cannot open the file manager

Предусловия	Пользователь не вошёл
-------------	-----------------------

Шаги

1. Открыть /files/

Ожидаемый результат

Адрес становится /login/?next=/files/, файлы не показаны.

Как работает автотест

- Как ТС-01, только для раздела файлов.
- Параметр `next=/files/` значит, что после входа приложение вернёт пользователя туда, куда он шёл.

Код: test_access.py

```
def test_11_anonymous_user_cannot_open_file_manager(page: Page):
    """TC-11: an anonymous user who opens /files/ is sent to sign in."""
    page.goto("/files/")

    expect(page).to_have_url(re.compile(r"/login/\\?next=/files/$"))
Запуск: pytest -k test_11 -v
```

ТС-12. Админ находит пользователя поиском

English: Admin finds a user by search

Предусловия	Выполнен вход как demo_admin. Есть демо-пользователи demo_user01...demo_user10
-------------	--

Шаги

1. Открыть /manage/
2. Ввести demo_user05 в поле поиска
3. Нажать «Search»

Ожидаемый результат

demo_user05 в списке. Других, например demo_user06, нет.

Как работает автотест

- `users_page.search(...)` заполняет поле поиска и нажимает кнопку.
- Позитивная проверка (нужный пользователь есть) плюс негативная (соседнего нет) — так видно, что поиск действительно фильтрует.

Код: test_users.py

```
def test_12_admin_finds_user_by_search(admin_page: Page):
    """TC-12: searching by username shows that user and hides the others."""
    users_page = UsersPage(admin_page).open()

    users_page.search("demo_user05")

    expect(users_page.row("demo_user05")).to_be_visible()
    expect(users_page.row("demo_user06")).to_have_count(0)
Запуск: pytest -k test_12 -v
```

ТС-13. Админ создаёт нового пользователя

English: Admin creates a new user

Предусловия	Выполнен вход как demo_admin
-------------	------------------------------

Шаги

1. Открыть /manage/users/new/
2. Ввести новое уникальное имя
3. Дважды ввести пароль E2e-Strong-Pass-42
4. Нажать «Create»

Ожидаемый результат

Видно сообщение «User <имя> created.», пользователь находится поиском в списке.

Как работает автотест

- `unique_name("user")` придумывает имя вида `e2e_user_3f9a1c2b`. Приложение не умеет удалять пользователей, поэтому каждый запуск создаёт нового — уникальное имя не даёт тестам столкнуться.
- `status_message` — зелёное сообщение приложения после действия, локатор описан в `BasePage`.
- `to_contain_text` проверяет, что текст содержит подстроку.

Код: test_users.py

```
def test_13_admin_creates_user(admin_page: Page):
    """TC-13: an admin creates a user and finds them in the list."""
    username = unique_name("user")
    create_page = UserCreatePage(admin_page).open()

    create_page.create_user(username, config.NEW_USER_PASSWORD)
```

```

users_page = UsersPage(admin_page)
expect(users_page.status_message).to_contain_text(f"User {username} created.")
users_page.search(username)
expect(users_page.row(username)).to_be_visible()

```

Запуск: `pytest -k test_13 -v`

ТС-14. Новый пользователь может войти

English: New user can sign in

Предусловия	Админ создал нового пользователя с известным паролем
-------------	--

Шаги

1. Открыть `/login/`
2. Ввести имя и пароль нового пользователя
3. Нажать «Sign in»

Ожидаемый результат

Открывается главная с «You are signed in as <имя>».

Как работает автотест

- Предусловие делает фикстура `new_user`: в отдельном браузере входит админом, создаёт пользователя и отдаёт тесту объект с `username` и `password`.
- Сам тест работает в своей вкладке `page` и просто входит этим пользователем.
- После теста фикстура отключает пользователя (код после `yield`).

Код: `test_users.py`

```

def test_14_new_user_can_sign_in(page: Page, new_user):
    """TC-14: a user created by an admin can sign in with their password."""
    login_page = LoginPage(page).open()

    login_page.sign_in(new_user.username, new_user.password)

    expect(HomePage(page).heading).to_have_text(
        f"You are signed in as {new_user.username}"
    )

```

Запуск: `pytest -k test_14 -v`

ТС-15. Пароль из одних цифр отклоняется

English: Password made only of digits is rejected

Предусловия	Выполнен вход как <code>demo_admin</code>
-------------	---

Шаги

1. Открыть `/manage/users/new/`
2. Ввести новое уникальное имя и дважды пароль `52012000`
3. Нажать «Create»
4. Найти это имя поиском в `/manage/`

Ожидаемый результат

Форма показывает «This password is entirely numeric.» Поиск ничего не находит: пользователь не создан.

Как работает автотест

- Негативный тест встроенных проверок пароля Django.
- Важно проверить не только текст ошибки, но и последствие: пользователь действительно не появился (`nothing_found` — «Nothing found.»).

Код: test_users.py

```
def test_15_numeric_password_is_rejected(admin_page: Page):
    """TC-15: a password made only of digits is rejected, no user is created."""
    username = unique_name("weak")
    create_page = UserCreatePage(admin_page).open()

    create_page.create_user(username, "52012000")

    expect(admin_page.get_by_text("This password is entirely numeric.")).to_be_visible()
    users_page = UsersPage(admin_page).open()
    users_page.search(username)
    expect(users_page.nothing_found()).to_be_visible()
Запуск: pytest -k test_15 -v
```

ТС-16. Отключённый пользователь больше не может войти

English: Deactivated user cannot sign in any more

Предусловия	Выполнен вход как demo_admin. Есть новый активный пользователь
-------------	--

Шаги

1. Открыть /manage/ и найти пользователя
2. Нажать «Deactivate» в его строке
3. В другом браузере попробовать войти этим пользователем

Ожидаемый результат

Видно «User <имя> disabled.», кнопка меняется на «Activate». Вход не удаётся.

Как работает автотест

- Используются три фикстуры: admin_page (админ), new_user (жертва) и new_context (из pytest-playwright).
- new_context().new_page() открывает второй, полностью отдельный браузер со своими cookie. В нём можно войти другим пользователем, не разлогинивая админа в первом.
- Проверяем и интерфейс админки (сообщение и кнопка), и реальный эффект (войти нельзя).

Код: test_users.py

```
def test_16_deactivated_user_cannot_sign_in(admin_page: Page, new_user, new_context):
    """TC-16: an admin switches a user off and that user can no longer sign in."""
    users_page = UsersPage(admin_page).open()
    users_page.search(new_user.username)

    users_page.deactivate_button(new_user.username).click()

    expect(users_page.status_message).to_contain_text(
        f"User {new_user.username} disabled."
    )
    expect(users_page.activate_button(new_user.username)).to_be_visible()

    # Try to sign in as that user in a separate browser.
    other_browser = new_context().new_page()
    login_page = LoginPage(other_browser).open()
    login_page.sign_in(new_user.username, new_user.password)
    expect(login_page.error_message).to_be_visible()
Запуск: pytest -k test_16 -v
```

ТС-17. Админ выдаёт роль admin

English: Admin grants the admin role

Предусловия	Выполнен вход как demo_admin. Есть новый пользователь без ролей
-------------	---

Шаги

1. Открыть /manage/ и нажать «Roles» в строке пользователя
2. Отметить «admin»
3. Нажать «Save»

Ожидаемый результат

Видно «Roles of <имя> updated», в строке пользователя есть роль «admin».

Как работает автотест

- open_roles_of(username) сначала ищет пользователя (список разбит на страницы по 10), затем жмёт «Roles» именно в его строке.
- role_checkbox("admin").check() ставит галочку. check() надёжнее click(): если галочка уже стоит, он её не снимет.
- Работаем с новым пользователем из фикстуры, а не с demo_user: если тест упадёт, общий демо-аккаунт не останется сломанным.

Код: test_roles.py

```
def test_17_admin_grants_admin_role(admin_page: Page, new_user):
    """TC-17: an admin grants the admin role and it shows in the user list."""
    users_page = UsersPage(admin_page).open()
    users_page.open_roles_of(new_user.username)
    roles_form = UserRolesPage(admin_page)

    roles_form.role_checkbox("admin").check()
    roles_form.save_button.click()

    expect(users_page.status_message).to_contain_text(
        f"Roles of {new_user.username} updated"
    )
    users_page.search(new_user.username)
    expect(users_page.role_badge(new_user.username, "admin")).to_be_visible()
```

Запуск: pytest -k test_17 -v

ТС-18. Админ снимает роль

English: Admin revokes a role

Предусловия	Выполнен вход как demo_admin. Есть новый пользователь с ролью «user»
-------------	--

Шаги

1. Открыть форму ролей пользователя
2. Снять отметку «user»
3. Нажать «Save»

Ожидаемый результат

В строке пользователя больше нет роли «user».

Как работает автотест

- Новый пользователь создаётся без ролей, поэтому тест сначала выдаёт ему «user» (это подготовка), а потом снимает (это сама проверка).
- uncheck() снимает галочку. Итог проверяем через to_have_count(0).

Код: test_roles.py

```
def test_18_admin_revokes_role(admin_page: Page, new_user):
    """TC-18: an admin takes a role away and it disappears from the user list."""
    users_page = UsersPage(admin_page).open()
    users_page.open_roles_of(new_user.username)
    roles_form = UserRolesPage(admin_page)
    roles_form.role_checkbox("user").check()
    roles_form.save_button.click()

    users_page.open_roles_of(new_user.username)
    roles_form.role_checkbox("user").uncheck()
    roles_form.save_button.click()

    users_page.search(new_user.username)
    expect(users_page.role_badge(new_user.username, "user")).to_have_count(0)
Запуск: pytest -k test_18 -v
```

ТС-19. «Cancel» в форме ролей ничего не меняет

English: Cancel on the roles form changes nothing

Предусловия	Выполнен вход как demo_admin. Есть новый пользователь без ролей
-------------	---

Шаги

1. Открыть форму ролей
2. Отметить «admin»
3. Нажать «Cancel» вместо «Save»

Ожидаемый результат

У пользователя по-прежнему нет роли «admin».

Как работает автотест

- Проверяем, что отмена действительно отменяет: галочку поставили, но форму не сохранили.

Код: test_roles.py

```
def test_19_cancel_on_roles_form_changes_nothing(admin_page: Page, new_user):
    """TC-19: leaving the roles form with "Cancel" keeps the roles as they were."""
    users_page = UsersPage(admin_page).open()
    users_page.open_roles_of(new_user.username)
    roles_form = UserRolesPage(admin_page)

    roles_form.role_checkbox("admin").check()
    roles_form.cancel_link.click()

    users_page.search(new_user.username)
    expect(users_page.role_badge(new_user.username, "admin")).to_have_count(0)
Запуск: pytest -k test_19 -v
```

ТС-20. Админ не может снять роль admin с самого себя

English: Admin cannot remove the admin role from themselves

Предусловия	Выполнен вход как demo_admin
-------------	------------------------------

Шаги

1. Открыть форму ролей demo_admin
2. Снять отметку «admin»
3. Нажать «Save»

Ожидаемый результат

Видна ошибка «You cannot take the admin role from yourself...». У demo_admin роль «admin» осталась.

Как работает автотест

- Защита от ситуации, когда админ случайно лишает себя доступа к панели.
- После проверки ошибки отдельно убеждаемся, что роль на месте. Это единственный тест, который трогает demo_admin, и он ничего не меняет в данных — приложение отклоняет сохранение.

Код: test_roles.py

```
def test_20_admin_cannot_remove_own_admin_role(admin_page: Page):
    """TC-20: an admin cannot take the admin role from themselves."""
    users_page = UsersPage(admin_page).open()
    users_page.open_roles_of(config.ADMIN.username)
    roles_form = UserRolesPage(admin_page)

    roles_form.role_checkbox("admin").uncheck()
    roles_form.save_button.click()

    expect(
        roles_form.error_message("You cannot take the admin role from yourself")
    ).to_be_visible()
    users_page.open()
    users_page.search(config.ADMIN.username)
    expect(users_page.role_badge(config.ADMIN.username, "admin")).to_be_visible()
```

Запуск: `pytest -k test_20 -v`

ТС-21. Изменение роли записывается в журнал аудита

English: Role change is written to the audit log

Предусловия	Выполнен вход как demo_admin. Есть новый пользователь без ролей
-------------	---

Шаги

1. Выдать новому пользователю роль «user»
2. Открыть /manage/audit/

Ожидаемый результат

Самая новая запись: кто — demo_admin, действие — granted, роль — user, кому — новый пользователь.

Как работает автотест

- newest_record в AuditPage — первая строка тела таблицы: журнал отсортирован от новых к старым.
- Четыре проверки — по одной на каждую колонку записи.
- Тест надёжен, потому что тесты идут по очереди: между выдачей роли и чтением журнала никто другой запись не добавит.

Код: test_roles.py

```
def test_21_role_change_is_written_to_audit_log(admin_page: Page, new_user):
    """TC-21: granting a role adds a record to the audit log: who, what, to whom."""
    users_page = UsersPage(admin_page).open()
    users_page.open_roles_of(new_user.username)
    roles_form = UserRolesPage(admin_page)
    roles_form.role_checkbox("user").check()
    roles_form.save_button.click()

    audit_page = AuditPage(admin_page).open()

    expect(audit_page.newest_record_cell(config.ADMIN.username)).to_be_visible()
    expect(audit_page.newest_record_cell("granted")).to_be_visible()
    expect(audit_page.newest_record_cell("user")).to_be_visible()
    expect(audit_page.newest_record_cell(new_user.username)).to_be_visible()
```

Запуск: `pytest -k test_21 -v`

ТС-22. Страница ролей учитывает нового участника

English: Roles page counts a new member

Предусловия	Выполнен вход как demo_admin. Есть новый пользователь без ролей
-------------	---

Шаги

1. Открыть /manage/roles/ и запомнить число пользователей в «admin»
2. Выдать новому пользователю роль «admin»
3. Снова открыть /manage/roles/

Ожидаемый результат

Число пользователей в «admin» выросло ровно на 1.

Как работает автотест

- Сначала запоминаем число «до» (members_count превращает текст ячейки в int).
- Ожидаем «до + 1». Сравниваем с текстом ячейки через to_have_text, поэтому число приводим к строке: str(admins_before + 1).

Код: test_roles.py

```
def test_22_roles_page_counts_new_member(admin_page: Page, new_user):
    """TC-22: granting the admin role adds 1 to its counter on the Roles page."""
    roles_page = RolesPage(admin_page).open()
    admins_before = roles_page.members_count("admin")

    users_page = UsersPage(admin_page).open()
    users_page.open_roles_of(new_user.username)
    roles_form = UserRolesPage(admin_page)
    roles_form.role_checkbox("admin").check()
    roles_form.save_button.click()

    roles_page.open()
    expect(roles_page.members_count_cell("admin")).to_have_text(str(admins_before + 1))
```

Запуск: pytest -k test_22 -v

ТС-23. Создать вложенную папку и вернуться по «хлебным крошкам»

English: Create a nested folder and come back by breadcrumbs

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Создать папку
2. Открыть её
3. Нажать имя рабочей папки в крошках
4. Нажать «Root» в крошках

Ожидаемый результат

1: папка в списке. 2: «Folder is empty.» 3: открыта рабочая папка, новая в ней. 4: открыт корень, рабочая папка в нём.

Как работает автотест

- Фикстура workspace уже создала и открыла пустую папку с уникальным именем и вернула это имя в переменную workspace.
- current_folder — элемент крошек с aria-current="page", то есть текущая папка.
- Тест длиннее остальных: навигация — это сценарий из нескольких шагов, и после каждого шага стоит своя проверка.

Код: test_folders.py

```
def test_23_create_nested_folder_and_go_back_by_breadcrumbs(
    files_page: FilesPage, workspace
):
    """TC-23: create a folder, open it, and come back through the breadcrumbs."""
    nested = unique_name("nested")

    files_page.create_folder(nested)
    expect(files_page.entry(nested)).to_be_visible()

    files_page.open_folder(nested)
    expect(files_page.current_folder).to_have_text(nested)
    expect(files_page.empty_folder_text).to_be_visible()

    files_page.breadcrumb(workspace).click()
    expect(files_page.current_folder).to_have_text(workspace)
    expect(files_page.entry(nested)).to_be_visible()

    files_page.breadcrumb("Root").click()
    expect(files_page.current_folder).to_have_text("Root")
    expect(files_page.entry(workspace)).to_be_visible()
Запуск: pytest -k test_23 -v
```

ТС-24. Папка с занятым именем не создаётся второй раз

English: Folder with a taken name is not created twice

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Создать папку
2. Ещё раз создать папку с тем же именем

Ожидаемый результат

Видно сообщение с «already exists». Папка с этим именем одна.

Как работает автотест

- to_have_count(1) проверяет, что дубликат не появился.

Код: test_folders.py

```
def test_24_folder_with_taken_name_is_not_created_twice(
    files_page: FilesPage, workspace
):
    """TC-24: a second folder with the same name is refused."""
    name = unique_name("folder")
    files_page.create_folder(name)

    files_page.create_folder(name)

    expect(files_page.status_message).to_contain_text("already exists")
    expect(files_page.entry(name)).to_have_count(1)
Запуск: pytest -k test_24 -v
```

ТС-25. Отмена удаления папки оставляет её на месте

English: Cancelling folder deletion keeps the folder

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Создать папку
2. Нажать «Delete» рядом с ней
3. На странице подтверждения нажать «Cancel»

Ожидаемый результат

Папка осталась в списке.

Как работает автотест

- `start_delete(name)` только нажимает «Delete», не подтверждая. Так тест может выбрать, что делать на странице подтверждения.

Код: test_folders.py

```
def test_25_cancelling_folder_deletion_keeps_folder(files_page: FilesPage, workspace):
    """TC-25: "Cancel" on the delete confirmation keeps the folder."""
    name = unique_name("folder")
    files_page.create_folder(name)

    files_page.start_delete(name)
    files_page.cancel_delete_link.click()
```

```
    expect(files_page.entry(name)).to_be_visible()
```

Запуск: `pytest -k test_25 -v`

ТС-26. Удаление папки удаляет её вместе с содержимым

English: Deleting a folder removes it with its contents

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Создать папку и загрузить в неё файл
2. Вернуться в рабочую папку и нажать «Delete» рядом с новой папкой
3. Нажать «Delete» на странице подтверждения

Ожидаемый результат

Шаг 2: предупреждение, что папка удалится со всем содержимым. Шаг 3: «Deleted: <имя>», папки нет.

Как работает автотест

- `make_file("inside.txt")` — фикстура-«фабрика»: она возвращает функцию, которая пишет файл во временную папку `pytest` и отдаёт путь к нему.
- `open_folder_by_path(workspace)` открывает рабочую папку прямо по адресу.
- Предупреждение проверяем до подтверждения — это часть ожидаемого результата.

Код: test_folders.py

```
def test_26_deleting_folder_removes_its_contents(
    files_page: FilesPage, workspace, make_file
):
    """TC-26: a folder with a file inside is deleted after a warning."""
    folder = unique_name("folder")
    files_page.create_folder(folder)
    files_page.open_folder(folder)
    files_page.upload(make_file("inside.txt"))
```

```

expect(files_page.entry("inside.txt")).to_be_visible()

files_page.open_folder_by_path(workspace)
files_page.start_delete(folder)
expect(files_page.folder_contents_warning).to_be_visible()
files_page.confirm_delete_button.click()

expect(files_page.status_message).to_contain_text(f"Deleted: {folder}.")
expect(files_page.entry(folder)).to_have_count(0)

```

Запуск: `pytest -k test_26 -v`

ТС-27. Переименование папки

English: Rename a folder

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Создать папку
2. Нажать «Rename» рядом с ней
3. Ввести новое имя и нажать «Save»

Ожидаемый результат

«Renamed to <новое имя>», в списке только новое имя.

Как работает автотест

- Проверяем три вещи: сообщение, новое имя есть, старого нет.

Код: `test_folders.py`

```

def test_27_rename_folder(files_page: FilesPage, workspace):
    """TC-27: a renamed folder shows only under its new name."""
    old_name = unique_name("before")
    new_name = unique_name("after")
    files_page.create_folder(old_name)

    files_page.rename(old_name, new_name)

    expect(files_page.status_message).to_contain_text(f"Renamed to {new_name}.")
    expect(files_page.entry(new_name)).to_be_visible()
    expect(files_page.entry(old_name)).to_have_count(0)

```

Запуск: `pytest -k test_27 -v`

ТС-28. Загрузка файла с именем на кириллице

English: Upload a file with a Cyrillic name

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Выбрать в «Upload files» текстовый файл с именем вроде «отчёт.txt»
2. Нажать «Upload»

Ожидаемый результат

«Uploaded files: 1.», файл в списке с правильным именем и типом «file».

Как работает автотест

- Кириллица в имени проверяет, что приложение не портит не-английские имена на всём пути: загрузка, хранение, отображение.
- `upload()` в `FilesPage` вызывает `set_input_files` — так Playwright выбирает файл без настоящего окна выбора файла.

- `entry_row(name)` — вся строка таблицы, в ней проверяем тип «file».

Код: `test_files.py`

```
def test_28_upload_file_with_cyrillic_name(files_page: FilesPage, workspace, make_file):
    """TC-28: an uploaded file appears in the list; a non-English name is kept."""
    name = f"{unique_name('отчёт')}.txt"

    files_page.upload(make_file(name, "report contents"))

    expect(files_page.status_message).to_contain_text("Uploaded files: 1.")
    expect(files_page.entry(name)).to_be_visible()
    expect(files_page.entry_row(name)).to_contain_text("file")
Запуск: pytest -k test_28 -v
```

ТС-29. Загрузка нескольких файлов за раз

English: Upload several files at once

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Выбрать сразу два текстовых файла в «Upload files»
2. Нажать «Upload»

Ожидаемый результат

«Uploaded files: 2.», оба файла в списке.

Как работает автотест

- `files_page.upload(first, second)` — метод принимает любое число файлов (*files в его определении).

Код: `test_files.py`

```
def test_29_upload_several_files_at_once(files_page: FilesPage, workspace, make_file):
    """TC-29: two files chosen together are both uploaded."""
    first = make_file("first.txt", "one")
    second = make_file("second.txt", "two")

    files_page.upload(first, second)

    expect(files_page.status_message).to_contain_text("Uploaded files: 2.")
    expect(files_page.entry("first.txt")).to_be_visible()
    expect(files_page.entry("second.txt")).to_be_visible()
Запуск: pytest -k test_29 -v
```

ТС-30. Скачанный файл совпадает с загруженным

English: Download returns the same file

Предусловия	Выполнен вход. В рабочую папку загружен файл с известным содержимым
-------------	---

Шаги

1. Нажать на имя файла

Ожидаемый результат

Браузер скачивает файл с тем же именем и тем же содержимым.

Как работает автотест

- `with page.expect_download() as download_info:` — сначала начинаем ждать скачивание, потом кликаем. Если поменять порядок, событие скачивания можно пропустить.

- `download.suggested_filename` — имя, которое предложил сервер, `download.path()` — путь к скачанному файлу на диске.
- Открываем файл обычным `open()` и сравниваем содержимое.

Код: `test_files.py`

```
def test_30_download_returns_same_file(files_page: FilesPage, workspace, make_file):
    """TC-30: a downloaded file has the same name and content as the uploaded one."""
    files_page.upload(make_file("download_me.txt", "useful payload"))

    # Start waiting for the download first, then click the file name.
    with files_page.page.expect_download() as download_info:
        files_page.entry("download_me.txt").click()
    download = download_info.value

    assert download.suggested_filename == "download_me.txt"
    with open(download.path(), encoding="utf-8") as downloaded:
        assert downloaded.read() == "useful payload"
```

Запуск: `pytest -k test_30 -v`

ТС-31. Переименование файла

English: Rename a file

Предусловия	Выполнен вход. В рабочую папку загружен файл
-------------	--

Шаги

1. Нажать «Rename» рядом с файлом
2. Ввести новое имя и нажать «Save»

Ожидаемый результат

В списке только новое имя.

Как работает автотест

- Как ТС-27, только для файла.

Код: `test_files.py`

```
def test_31_rename_file(files_page: FilesPage, workspace, make_file):
    """TC-31: a renamed file shows only under its new name."""
    files_page.upload(make_file("before.txt"))

    files_page.rename("before.txt", "after.txt")

    expect(files_page.entry("after.txt")).to_be_visible()
    expect(files_page.entry("before.txt")).to_have_count(0)
```

Запуск: `pytest -k test_31 -v`

ТС-32. Удаление файла после подтверждения

English: Delete a file after confirmation

Предусловия	Выполнен вход. В рабочую папку загружен файл
-------------	--

Шаги

1. Нажать «Delete» рядом с файлом
2. Нажать «Delete» на странице подтверждения

Ожидаемый результат

«Deleted: <имя>», файла нет в списке.

Как работает автотест

- `files_page.delete(name)` делает оба шага: нажимает «Delete» и подтверждает.

Код: `test_files.py`

```
def test_32_delete_file_after_confirmation(files_page: FilesPage, workspace, make_file):
    """TC-32: a file is deleted after confirming."""
    files_page.upload(make_file("delete_me.txt"))

    files_page.delete("delete_me.txt")

    expect(files_page.status_message).to_contain_text("Deleted: delete_me.txt.")
    expect(files_page.entry("delete_me.txt")).to_have_count(0)
Запуск: pytest -k test_32 -v
```

ТС-33. Файл больше лимита отклоняется

English: File over the size limit is rejected

Предусловия	Выполнен вход. Открыта пустая рабочая папка. Лимит известен (5 МБ в docker-compose)
-------------	---

Шаги

1. Выбрать файл на 1 КБ больше лимита
2. Нажать «Upload»

Ожидаемый результат

Видно сообщение с «is larger than the allowed». Файла нет в списке.

Как работает автотест

- Лимит берётся из `config.MAX_FILE_SIZE` (переменная окружения `E2E_MAX_FILE_SIZE`). Если в приложении другой лимит, поменяйте переменную, а не тест.
- `tmp_path` — встроенная фикстура `pytest`: временная папка, которую `pytest` сам удалит. Файл нужного размера пишем через `write_bytes`.

Код: `test_files.py`

```
def test_33_file_over_size_limit_is_rejected(
    files_page: FilesPage, workspace, tmp_path
):
    """TC-33: a file bigger than the limit is refused with a clear message."""
    big_file = tmp_path / "too_big.bin"
    big_file.write_bytes(b"0" * (config.MAX_FILE_SIZE + 1024))

    files_page.upload(big_file)

    expect(files_page.status_message).to_contain_text("is larger than the allowed")
    expect(files_page.entry("too_big.bin")).to_have_count(0)
Запуск: pytest -k test_33 -v
```

ТС-34. Файл с занятым именем не перезаписывается

English: File with a taken name is not overwritten

Предусловия	Выполнен вход. Открыта пустая рабочая папка в /files/
-------------	---

Шаги

1. Загрузить `same_name.txt` с текстом «first version»
2. Загрузить другой `same_name.txt` с текстом «second version»
3. Скачать `same_name.txt`

Ожидаемый результат

На шаге 2 сообщение с «already exists». Скачанный файл содержит «first version».

Как работает автотест

- Два файла с одинаковым именем не могут лежать в одной папке на диске, поэтому тест кладёт их в подпапки v1 и v2 временной папки.
- Главная проверка — содержимое: старый файл не подменился новым.

Код: test_files.py

```
def test_34_file_with_taken_name_is_not_overwritten(
    files_page: FilesPage, workspace, tmp_path
):
    """TC-34: uploading a file with an existing name keeps the old file."""
    first_version = tmp_path / "v1" / "same_name.txt"
    second_version = tmp_path / "v2" / "same_name.txt"
    first_version.parent.mkdir()
    second_version.parent.mkdir()
    first_version.write_text("first version", encoding="utf-8")
    second_version.write_text("second version", encoding="utf-8")
    files_page.upload(first_version)

    files_page.upload(second_version)

    expect(files_page.status_message).to_contain_text("already exists")
    with files_page.page.expect_download() as download_info:
        files_page.entry("same_name.txt").click()
    with open(download_info.value.path(), encoding="utf-8") as downloaded:
        assert downloaded.read() == "first version"
```

Запуск: `pytest -k test_34 -v`

5. Как написать свой тест

Возьмём кейс, которого ещё нет: «админ снова включает отключённого пользователя».

1. **Опишите кейс.** Добавьте запись TC-35 в `tools/test_cases.json`: название, предусловия, шаги, ожидаемый результат, имя будущего теста.
2. **Найдите похожий тест.** Ближе всего TC-16 (отключение) в `tests/test_users.py`. Скопируйте его и переименуйте в `test_35_admin_activates_user`.
3. **Подготовка.** Нужен отключённый пользователь: возьмите `new_user` и нажмите `deactivate_button`.
4. **Действие.** Нажмите `activate_button(...)` — он уже есть в `UsersPage`.
5. **Проверки.** Сообщение `User <имя> enabled.`, кнопка снова «Deactivate», и в отдельном браузере пользователь может войти.
6. **Запустите.** `pytest -k test_35 --headed --slowmo 500` — посмотрите глазами, что происходит.
7. **Убедитесь, что тест умеет падать.** Временно поменяйте ожидание (например, `enabled` на `disabled`) — тест должен покраснеть. Верните.
8. **Линтеры и коммит.** `black ., ruff check --fix .`, затем `python tools/create_test_case_issues.py --markdown` и КОММИТ.

```
def test_35_admin_activates_user(admin_page: Page, new_user, new_context):
    """TC-35: an admin switches a user back on and they can sign in."""
    users_page = UsersPage(admin_page).open()
    users_page.search(new_user.username)
    users_page.deactivate_button(new_user.username).click()

    users_page.activate_button(new_user.username).click()

    expect(users_page.status_message).to_contain_text(
        f"User {new_user.username} enabled."
    )
    other_browser = new_context().new_page()
    LoginPage(other_browser).open().sign_in(new_user.username, new_user.password)
    expect(HomePage(other_browser).heading).to_be_visible()
```

Если нужного элемента нет в Page Object

Запустите `playwright codegen http://127.0.0.1:8000`, покликайте мышкой — Playwright покажет локатор. Перенесите его в нужный файл `pages/` как атрибут или метод, а в тесте используйте уже его. Сгенерированный код целиком в тест не копируйте: в нём лишние `click()` перед `fill()` и нет проверок.

Частые ошибки новичка

- `time.sleep(2)` «чтобы страница успела». Вместо этого — `expect(...)`, он ждёт сам.
- Проверка через `if element.is_visible()`: — не ждёт и может соврать. В тестах используйте `expect`.
- Изменение общих демо-аккаунтов. Создавайте свои данные через `new_user`, `workspace`, `unique_name`.
- Один огромный тест «на всё». Один тест — один кейс: когда он упадёт, по имени сразу понятно, что сломалось.
- Тест, который ни разу не падал. Всегда проверьте, что он краснеет, если сломать ожидание.

6. Рекомендации: как учиться на этом проекте

1. **Читайте в таком порядке:** `config.py` → `pages/base_page.py` → `pages/login_page.py` → `tests/test_login.py` → `conftest.py` → остальные тесты. От простого к сложному.
2. **Запускайте с** `--headed --slowmo 500`. Смотрите, что делает браузер, и сверяйте с кодом строка за строкой.
3. **Ломайте специально.** Поменяйте пароль в `config.py`, текст в `expect`, имя кнопки в `Page Object`. Посмотрите, как выглядит падение и трейс — это главный навык при разборе упавших тестов.
4. **Пишите сами по одному кейсу.** Идеи: TC-35 (включение пользователя, см. раздел 5), выход по «Sign out» из админки, поиск без результатов, пустое имя папки, вход админом и переход по ссылке «Users» в шапке.
5. **Каждый новый кейс — через issue → ветку → PR**, как на работе. Даже если вы единственный участник, это привычка, которую спросят на собеседовании.
6. **Следующий шаг — CI (issue #4).** GitHub Actions поднимает приложение в Docker и запускает `pytest` на каждый PR. Готовый пример — job «E2E (Playwright)» в `.github/workflows/ci.yml` приложения: там только заменить `npx playwright test` на `pytest`.
7. **Потом — отчёты.** Плагин `pytest-html` или `Allure` дают красивый отчёт о прогоне; добавьте его, когда освоитесь с основой.